

# Provisional Application Cover Sheet

This submission is identified as a provisional application for patent under 35 U.S.C. 111(b) and 37 C.F.R. 1.51(c)(1).

## Title of Invention

Encrypted Neural Embedding Relay for Privacy-Preserving Neuroinference

## Inventor Information

Inventor legal name(s)

1. \_\_\_\_\_
2. \_\_\_\_\_
3. \_\_\_\_\_

Inventor residence(s), including city, state, and country

1. \_\_\_\_\_
2. \_\_\_\_\_
3. \_\_\_\_\_

## Applicant / Ownership Information

Applicant name, if different from inventor: \_\_\_\_\_

Ownership or assignment information, if applicable: \_\_\_\_\_

## Correspondence Address

Name: \_\_\_\_\_  
Street address: \_\_\_\_\_  
City, state, postal code, country: \_\_\_\_\_  
Email: \_\_\_\_\_  
Telephone: \_\_\_\_\_

## Attorney, Agent, or Docket Information

Attorney or agent name, if applicable: \_\_\_\_\_  
USPTO registration number, if applicable: \_\_\_\_\_  
Docket number, if applicable: \_\_\_\_\_

## Government Interest

☐ No ☐ Yes. If yes, identify the agency and interest: \_\_\_\_\_

## Enclosed Application Papers

☐ Specification: ENER\_provisional\_specification.pdf ☐ Drawings: ENER\_drawings.pdf ☐ Filing fee

☐ Micro entity certification, if applicable ☐ Other: \_\_\_\_\_

## Entity Status for Fee Purposes

☐ Large entity ☐ Small entity ☐ Micro entity

## Signature

Signature: \_\_\_\_\_  
Printed name: \_\_\_\_\_  
Date: \_\_\_\_\_

# Encrypted Neural Embedding Relay for Privacy-Preserving Neuroinference

Inventor drafting package: U.S. provisional utility application specification

Project codename: ENER

Draft status: Technical provisional draft

## Invention Disclosure Summary

### Problem Statement

Neural telemetry from brain-computer interfaces, neurotechnology wearables, implanted arrays, and related sensing devices can contain information about motor intent, attention, fatigue, affect, speech intent, impairment, identity, and other cognition-adjacent states. Conventional privacy architectures generally encrypt data during transport or storage, but often decrypt raw signals or high-dimensional features in a remote processor for inference. This exposes sensitive neural material to cloud services, model hosts, logging systems, analytics vendors, or compromised infrastructure.

The technical problem addressed by this disclosure is how to permit useful neuroinference while preventing raw neural telemetry and reconstruction-capable neural features from being exposed to remote compute environments. The disclosed system compresses neural telemetry into task-sufficient latent embeddings at an edge device, encrypts the latent embeddings before external transmission, and performs selected inference operations over the encrypted latent-space representation.

### Technical Field

The disclosure relates to privacy-preserving neural signal processing, brain-computer interfaces, neurotechnology telemetry, edge computing, compressed latent representations, homomorphic encryption, secure multiparty computation, trusted execution environments, and encrypted machine-learning inference.

### Background

Neural acquisition devices can include EEG headsets, electrocorticography arrays, implanted microelectrode arrays, magnetoencephalography systems, functional near-infrared spectroscopy devices, wearable neurotechnology, and hybrid biosignal devices. Such systems commonly segment raw time-series signals into windows, extract features, and classify states such as motor intent, attention, workload, seizure risk, cognitive fatigue, imagined speech, or user commands.

Remote compute is attractive because neural decoding models may require specialized processors, frequent updates, high-throughput batch inference, clinical collaboration, or integration with cloud applications. However, sending raw neural telemetry or plaintext neural features to a remote model creates a serious privacy exposure. Even if the channel is encrypted during transport, remote inference services may receive plaintext after decryption.

### Limitations of Prior EEG Encryption and Related Approaches

Known approaches to neural-data security often focus on encrypting raw EEG files, encrypting streams during transmission, access-controlling stored recordings, or applying privacy policies to server-side data. Such approaches can be useful, but they leave several gaps:

- Raw-signal encryption protects storage or transmission, but remote inference may still require decryption before computation.
- Encrypting raw EEG or other dense neural telemetry can impose high ciphertext expansion, latency, and energy cost.
- Raw neural traces and high-dimensional features may be reconstruction-rich, linkable, and reusable for future inferences beyond the original task.
- Conventional feature extraction may not optimize for homomorphic-encryption depth, ciphertext packing, bandwidth, entropy, or neural privacy risk.
- Prior systems may not adapt latent dimensionality or quantization based on encryption budget, network bandwidth, signal quality, or inference target.
- De-identification is fragile because neural features can be individualized, longitudinal, and linkable to identity, impairment, or behavior.

### Summary of Invention

An encrypted neural embedding relay receives neural telemetry at or near a user, segments the telemetry into windows, and uses a local encoder to generate a compressed latent embedding. The encoder may be trained or configured to preserve task-relevant information while reducing reconstruction of raw cognitive or neural signals. The embedding is compressed, quantized, sparsified, packed, or otherwise shaped for privacy-preserving computation. An encryption module encrypts the latent embedding using homomorphic encryption or related cryptographic methods. A relay interface transmits encrypted latent representations to a compute environment that performs inference over encrypted embeddings. The system returns an encrypted result, a decrypted result to an authorized party, a proof, or a policy decision without exposing raw neural telemetry to the remote processor.

In a presently preferred embodiment, spike-event or event-derived neural telemetry is processed by a spatial spike sorter within a trusted local boundary. The spatial spike sorter converts a temporal neural event window into sorted sparse events. A permitted descriptor of active neuron, channel, time-bin, or spatial-bin positions may be transmitted in public, padded, coarsened, or encrypted form according to a privacy policy. Active feature values are encrypted before leaving the local boundary. A remote encrypted inference engine evaluates a depth-1 linear score contract over the encrypted active feature values, such as  $\text{scores} = W \times x + \text{bias}$ , using BFV, BFVrns, BGV, TFHE-style integer operations, or a related protected arithmetic mode. The remote engine therefore receives enough structure to perform the selected score computation, but does not receive raw neural telemetry or plaintext active feature values.

The system may use fully homomorphic encryption, somewhat homomorphic encryption, leveled homomorphic encryption, secure multiparty computation, trusted execution environments, differential privacy, secure enclaves, hybrid public-key and symmetric encryption, post-quantum key establishment, or combinations thereof. Compression may adapt dynamically based on encryption cost, bandwidth, entropy, signal quality, device power, inference target, privacy budget, and model confidence.

### **Advantages**

- Raw neural telemetry can remain on a trusted edge device, local server, mobile device, or secure enclave.
- Remote processors can perform useful inference on encrypted latent embeddings without viewing raw neural signals.
- Latent compression can reduce ciphertext size, homomorphic operation count, network bandwidth, and inference latency.
- Reconstruction-resistant embeddings can reduce the risk that encrypted or later-decrypted features reveal unnecessary cognitive content.
- Adaptive compression can tune dimensionality, quantization, sparsity, and privacy parameters according to available compute and task requirements.
- Split-device neuroprocessing can separate acquisition, encoding, encryption, inference, and output authorization across different trust zones.
- The architecture can support many sensor modalities and deployment environments.

### **Commercial Applications**

Potential applications include assistive brain-computer interfaces, prosthetic-control systems, attention and workload tools, privacy-preserving neurofeedback, consumer neurotechnology, secure clinical decision support, hospital or research collaboration, encrypted motor-intent classification, encrypted speech-intent support, privacy-preserving neurodiagnostic screening, human-computer interaction, gaming interfaces, defense or aerospace fatigue monitoring, workplace safety systems with local-first privacy, and multi-user encrypted neural analytics.

### **Definitions**

"Neural telemetry" means raw, preprocessed, or derived signals generated from central or peripheral nervous-system activity, including EEG, ECoG, fNIRS, MEG, implanted-array signals, local field potentials, spike trains, wearable BCI signals, or hybrid biosignals.

"Edge device" means a device located within a trusted or semi-trusted local boundary, including a headset, implant controller, phone, tablet, wearable hub, laptop, home server, clinical workstation, vehicle system, or local gateway.

"Latent embedding" means a compressed representation produced by an encoder from a raw or

intermediate neural signal window. The embedding may be a vector, tensor, token sequence, spike/event list, low-bit code, sparse feature set, learned representation, random projection, quantized feature, or packed ciphertext input.

"Reconstruction-resistant" means configured, trained, constrained, or tested to reduce reconstruction of raw neural telemetry or sensitive cognitive content from the embedding while retaining task utility.

"Encrypted inference" means inference performed over encrypted data, protected shares, secure-enclave data, or a hybrid protected representation such that a remote processor does not receive raw neural telemetry in plaintext.

"Relay" means a communication, translation, packaging, routing, policy, or orchestration layer that moves protected embeddings between a local device and an inference engine.

"Privacy budget" means a system-level parameter or set of parameters controlling acceptable information exposure, including differential-privacy noise, latent dimensionality, metadata exposure, quantization level, padding, batching, retention, or reconstruction-risk thresholds.

"Spatial spike sorter" means a local hardware, firmware, software, or mixed-signal component that receives spike-event or event-derived telemetry and emits a sorted sparse event representation keyed by time bin, neuron or channel identifier, spatial bin, and active feature value.

"Permitted position descriptor" means metadata describing active positions of a sparse representation, such as active neuron identifiers, channel identifiers, time bins, spatial bins, or flattened feature indices, that may be transmitted in public, padded, coarsened, randomized, or encrypted form according to a privacy policy. A permitted position descriptor does not include plaintext active feature values unless the selected policy expressly allows them to remain local or public.

"Depth-1 encrypted linear scorer" means an encrypted inference component that evaluates a bounded-depth score function such as  $\text{scores} = W \times + \text{bias}$ , in which encrypted inputs are multiplied by plaintext, encrypted, secret-shared, or enclave-protected weights and accumulated into encrypted class scores with a single multiplicative layer.

## **A. Title**

Encrypted Neural Embedding Relay for Privacy-Preserving Neuroinference

## **B. Technical Field**

This disclosure relates to systems, methods, and computer-readable media for privacy-preserving processing of neural telemetry. More particularly, the disclosure relates to local latent encoding of neural signals, compression of neural representations before encryption, and encrypted inference over latent-space embeddings using homomorphic encryption or related privacy-preserving computation.

## **C. Background**

Brain-computer interfaces and neurotechnology systems acquire sensitive neural or neuro-adjacent signals. Such signals may originate from non-invasive EEG electrodes, ECoG grids, implanted arrays, fNIRS optodes, MEG sensors, wearable headsets, peripheral nerve interfaces, or hybrid biosignal platforms. A processing pipeline may segment the signal into temporal windows, remove artifacts, extract features, and classify cognitive or motor states.

Many useful pipelines depend on remote processors, centralized model hosting, cloud accelerators, multi-user analytics, or collaboration across clinical and research sites. In a conventional architecture, the device encrypts the stream in transit, sends the data to a server, decrypts the stream for inference, and stores or logs intermediate representations. This architecture is poorly suited to neural telemetry because the signal can be intimate, involuntary, individually identifying, and useful for future inferences that were not known at the time of collection.

Homomorphic encryption and related cryptographic methods can allow selected computation on encrypted data. However, directly applying encrypted computation to raw neural streams is often impractical because raw streams may be dense, high-dimensional, noisy, and expensive to process under encrypted arithmetic. Further, even encrypted raw-signal storage does not address whether the data should be collected, transmitted, or retained at full fidelity.

There is a need for a split-device neuroprocessing architecture in which raw neural telemetry is transformed locally into compressed, task-oriented, reconstruction-resistant latent embeddings before encryption, and in which remote processors perform useful inference without seeing raw neural telemetry or plaintext decode-capable features.

## **D. Summary**

In one embodiment, a neural acquisition device receives neural telemetry from one or more sensors. An edge latent encoder segments the telemetry into windows and generates a compressed latent embedding for each window. The encoder may include signal-conditioning layers, artifact rejection, filtering, normalization, temporal convolution, recurrent processing, transformer layers, spiking neural network layers, autoencoder layers, random projections, wavelet features, event extraction, or combinations thereof.

An adaptive compression controller selects a compression policy based on signal quality, encryption budget, bandwidth, entropy, task type, latency target, device power, privacy budget, or model confidence. The compression policy may control latent dimensionality, quantization level, sparsity, packing layout, event threshold, window length, overlap, noise injection, padding, batching, or whether a particular computation is performed locally rather than remotely.

An encryption module encrypts the compressed latent embedding using a privacy-preserving cryptographic mode. In examples, the module uses fully homomorphic encryption, leveled homomorphic encryption, somewhat homomorphic encryption, BFV, BGV, CKKS, TFHE-style gate bootstrapping, multi-key homomorphic encryption, secure multiparty computation, threshold encryption, secret sharing, trusted execution environments, secure enclaves, differential privacy, hybrid encryption, or post-quantum transport wrapping.

A relay interface transmits the encrypted latent embedding to an encrypted inference engine. The inference engine applies one or more model layers, scoring functions, classifiers, filters, policy checks, verifiers, or decision functions to the encrypted latent representation. The inference engine may output encrypted logits, encrypted scores, an encrypted class label, a decrypted class label to an authorized endpoint, a confidence score, a proof of computation, a control signal, or an audit record. At no point is the remote inference engine required to receive raw neural telemetry in plaintext.

The architecture supports EEG, ECoG, fNIRS, MEG, implanted arrays, wearable BCI devices, edge devices, cloud servers, local servers, mobile devices, secure enclaves, and hybrid edge-cloud deployments. It also supports multiple inference tasks, including motor-intent classification, attention-state classification, fatigue detection, neurofeedback, communication assistance, medical or neurodiagnostic inference, anomaly detection, and privacy-preserving research analytics.

## **E. Brief Description of Drawings**

Figure 1 illustrates an overall encrypted neural embedding relay pipeline in which a neural sensor provides telemetry to an edge encoder, a compressed latent embedding is encrypted, and encrypted inference produces an output without exposing raw neural telemetry.

Figure 2 illustrates a split-device architecture including a BCI headset or neural acquisition device, a phone or edge hub, an encrypted compute server, and a client application.

Figure 3 illustrates an adaptive latent compression controller that receives signal quality, bandwidth, encryption budget, and inference target information and outputs a compression policy.

Figure 4 illustrates reconstruction-resistant encoder training in which an encoder is optimized for task utility while an adversarial decoder is blocked, penalized, or constrained from reconstructing raw neural signals.

Figure 5 illustrates an encrypted cognitive-state classifier in which an encrypted latent vector is processed by encrypted model layers to produce encrypted or authorized classification results.

Figure 6 illustrates a spatial sparse-event embodiment in which a local spatial spike sorter emits public, padded, coarsened, or encrypted active-position descriptors and encrypted active feature values for a BFVrns depth-1 linear scorer.

## **F. Detailed Description**

## F.1 System Overview

The disclosed encrypted neural embedding relay may include a neural acquisition subsystem, an edge preprocessing subsystem, an edge latent encoder, an adaptive compression controller, a cryptographic module, a relay interface, an encrypted inference engine, a decryption or authorization endpoint, and an audit or policy subsystem.

The neural acquisition subsystem samples neural telemetry from one or more sensors. The telemetry may include voltage measurements, optical measurements, magnetic measurements, spike events, local field potentials, spectral features, event-related potentials, hemodynamic signals, artifact markers, device motion information, impedance measurements, stimulation context, or calibration metadata. The acquisition subsystem may be integrated into a headset, implant controller, wearable, clinical device, mobile device, or local server.

The edge preprocessing subsystem may apply filtering, artifact rejection, resampling, normalization, referencing, windowing, denoising, baseline correction, event extraction, channel selection, quality scoring, or synchronization with other local sensors. The subsystem may segment the telemetry into windows such as 10 ms, 50 ms, 100 ms, 250 ms, 1 second, overlapping windows, event-triggered windows, session-specific windows, or task-specific windows.

The edge latent encoder converts each window into a latent embedding. The latent embedding may be lower-dimensional than the raw window. The embedding may contain integer, binary, ternary, fixed-point, floating-point, sparse, packed, or polynomial-ring-compatible values. The encoder may be trained to preserve target information such as motor intent while suppressing unnecessary information such as identity, raw waveform details, or task-irrelevant cognitive state.

The cryptographic module encrypts the compressed latent embedding before it is transmitted outside the trusted edge boundary. The encrypted inference engine then computes over the encrypted embedding. Depending on implementation, the inference engine may use plaintext model weights with encrypted inputs, encrypted model weights with encrypted inputs, secret-shared model weights, secure-enclave-protected model weights, or hybrid execution across multiple parties.

## F.2 Local Segmentation and Windowing

Raw neural telemetry may be segmented into windows using a fixed sample count, fixed time duration, event-triggered boundary, task-state boundary, physiological boundary, or adaptive boundary. For EEG, a window may include multiple electrode channels sampled at rates such as 128 Hz, 256 Hz, 512 Hz, 1 kHz, or other rates. For implanted arrays, a window may include spike counts, threshold crossings, local field potential features, or binned firing rates. For fNIRS, a window may include oxygenated and deoxygenated hemoglobin features over longer time scales. For MEG, a window may include source-localized or sensor-level magnetic activity.

The system may apply overlap between adjacent windows to maintain temporal resolution. Window metadata may include time, device identifier, calibration version, channel mask, signal quality, impedance, sampling rate, artifact flags, and task context. Privacy controls may decide which metadata remains local, which metadata is encrypted, which metadata is coarsened, and which metadata may be transmitted in plaintext.

## F.3 Latent Encoder

The latent encoder may be implemented as a neural network, digital signal processing pipeline, spiking encoder, autoencoder, variational autoencoder, contrastive encoder, transformer encoder, temporal convolutional network, recurrent network, reservoir computer, random projection module, wavelet transform, spectral feature extractor, or hybrid of learned and deterministic stages.

The encoder may output:

- A dense vector of latent values.
- A sparse vector with active indices and values.
- A sequence of tokens representing neural events.
- A spike/event tensor.
- A low-bit integer feature vector.
- A binary hash or locality-sensitive code.
- A quantized embedding packed for homomorphic encryption slots.
- A secret-shared embedding for multiparty computation.

The encoder may be personalized to a user or generalized across users. Personalization may occur locally using calibration data that does not leave the trusted boundary. Model updates may be transferred using federated learning, encrypted gradients, secure aggregation, differential privacy, or update filtering.

### F.3.1 Spatial Spike Sorter Embodiment

In a preferred event-derived embodiment, the local encoder includes a spatial spike sorter. The sorter receives an event window containing spike counts, threshold crossings, binned firing activity, event-camera-adjacent signals, or other sparse neural or neuro-adjacent events. The sorter maps each event to a canonical position tuple such as:

```
event = {  
    index: flattened feature index,  
    timeBin: local time bin,  
    neuronId: local neuron or channel identifier,  
    unitX: spatial bin x-coordinate,  
    unitY: spatial bin y-coordinate,  
    value: active feature value  
}
```

The sorter may be implemented as FPGA logic, an ASIC block, a neuromorphic front end, an implant-controller routine, a headset controller, a secure-enclave routine, a mobile digital-signal-processing stage, or an edge finite-state machine. The sorter output may separate position metadata from feature values. Position metadata may be public, padded, coarsened, randomized, encrypted, or withheld based on a selected privacy mode. Active feature values are encrypted before being transmitted outside the trusted local boundary.

This embodiment supports a lightweight encrypted linear path in which the encrypted inference engine evaluates active events directly. Because inactive zero features need not be encrypted in the fastest sparse mode, the remote scorer can avoid encrypted operations for inactive positions while still returning encrypted class scores. A denser mode may encrypt inactive and active positions alike when the position pattern itself is sensitive.

### F.4 Compression Before Encryption

Compression before encryption is a central feature of the disclosed system. Rather than encrypting raw neural telemetry and then attempting remote compression or inference, the edge device first reduces the signal to a task-sufficient latent representation. This can lower bandwidth, ciphertext expansion, homomorphic multiplicative depth, bootstrapping frequency, memory use, and compute latency.

Compression may include dimensionality reduction, channel selection, temporal pooling, event thresholding, quantization, sparsification, entropy coding, learned bottlenecks, pruning, hashing, binarization, delta encoding, spike counting, time-to-first-spike encoding, low-rank projection, or ciphertext-slot packing. Compression may be constrained to retain a target inference accuracy, confidence threshold, calibration stability, or safety margin.

In one embodiment, the encoder produces a latent vector of 32 to 512 values from an input window containing thousands of raw samples. In another embodiment, the encoder produces a sparse active-event list with encrypted values and optionally public, padded, or encrypted active positions. In another embodiment, the encoder produces a packed polynomial representation configured for BFV, BGV, or CKKS evaluation.

### F.5 Adaptive Compression Controller

The adaptive compression controller may select a compression policy at runtime. Inputs to the controller may include:

- Signal quality metrics such as signal-to-noise ratio, channel impedance, artifact level, motion contamination, dropout, or confidence.
- Bandwidth metrics such as uplink rate, packet loss, latency, congestion, or local network availability.
- Encryption budget metrics such as ciphertext size, multiplicative depth, noise budget, bootstrapping allowance, key-switching cost, rotation count, and available memory.
- Device metrics such as battery level, processor load, thermal limit, secure enclave

availability, and local accelerator availability.

- Task metrics such as motor-intent classification, attention-state classification, anomaly detection, medical screening, speech-intent inference, or control-loop urgency.
- Privacy metrics such as reconstruction-risk score, differential-privacy budget, metadata leakage score, identity-obfuscation target, and policy constraints.

The controller may output window length, latent dimension, quantization precision, sparsity threshold, packing layout, encryption scheme, local-vs-remote routing, padding level, batch size, noise level, and whether to reject or defer inference. For example, when bandwidth is constrained and signal quality is high, the controller may use a smaller latent dimension. When reconstruction risk is high, the controller may increase adversarial suppression, lower precision, add noise, or keep inference local. When the encryption noise budget is low, the controller may select a shallower model or a lower-depth polynomial approximation.

## **F.6 Encryption and Privacy-Preserving Computation**

The cryptographic module may implement one or more of the following:

- Fully homomorphic encryption for arbitrary or broad classes of computation over encrypted embeddings.
- Leveled or somewhat homomorphic encryption for bounded-depth inference circuits.
- CKKS-style approximate arithmetic for real-valued or fixed-point latent vectors.
- BFV or BGV-style exact modular arithmetic for integer, low-bit, or spike-count features.
- TFHE-style Boolean or lookup-table operations for threshold classifiers.
- Multi-key FHE for embeddings encrypted under different users' keys.
- Secure multiparty computation using secret sharing, garbled circuits, or oblivious transfer.
- Trusted execution environments or secure enclaves for protected local or server-side inference.
- Differential privacy for aggregate training, telemetry summaries, or released model updates.
- Hybrid encryption in which symmetric encryption protects transport and storage while FHE, MPC, or enclaves protect computation.
- Post-quantum key establishment and signatures for transport, identity, artifact integrity, or relay policy.

The system may choose cryptographic parameters based on security level, latency, model depth, ciphertext packing, user class, regulatory context, or device capability. The system may maintain a cryptographic inventory identifying which algorithms protect transport, storage, identity, signatures, encrypted computation, and key custody.

## **F.7 Encrypted Inference Engine**

The encrypted inference engine may evaluate linear classifiers, logistic-regression approximations, neural network layers, polynomial activations, decision trees, lookup tables, support vector machines, nearest-centroid models, temporal filters, anomaly detectors, policy rules, or ensemble models. Operations may include encrypted addition, encrypted multiplication, plaintext-ciphertext multiplication, rotations, rescaling, relinearization, bootstrapping, comparison approximations, packed SIMD operations, and encrypted argmax approximations.

In some embodiments, the model weights are public while the latent embedding remains encrypted. In other embodiments, model weights are encrypted or secret-shared to protect proprietary models or reduce inference leakage. In still other embodiments, only a first stage is encrypted and a later stage is performed locally after partial decryption by the authorized device.

The result may be an encrypted score vector, encrypted class label, confidence interval, encrypted anomaly flag, encrypted motor-intent command, encrypted attention-state estimate, or encrypted medical/neurodiagnostic output. Decryption may occur only at an authorized client, local hub, clinician workstation, patient-controlled device, or threshold set of parties.

## **F.8 Reconstruction-Resistant Embeddings**

The latent encoder may be trained or configured to resist reconstruction of raw neural telemetry. Reconstruction resistance may be achieved using one or more of the following:

- Bottlenecked latent dimensionality.
- Quantization or binarization.
- Sparsity constraints.
- Adversarial training against a decoder attempting to reconstruct raw signals.



- Identity-adversarial loss to reduce user identification from embeddings.
- Mutual-information penalties between embeddings and raw signal details not needed for the task.
- Differential-privacy noise.
- Randomized projections or session-specific transformations.
- Channel dropout or task-specific masking.
- Encoder certification tests measuring reconstruction error, identity leakage, or sensitive-attribute leakage.

An adversarial decoder may receive embeddings and attempt to reconstruct raw windows, identity labels, session labels, or sensitive attributes. The encoder is optimized to maintain target task performance while reducing decoder success. The system may reject an encoder version if reconstruction-risk metrics exceed a configured threshold.

In one training procedure, an encoder E, task head T, reconstruction decoder D, and identity adversary A are trained on local or governed training data. The decoder is trained to minimize:

$$L_{\text{decoder}} = L_1(\text{raw\_window}, D(E(\text{raw\_window}))) + \alpha * \text{spectral\_loss}(\text{raw\_window}, D(E(\text{raw\_window})))$$

The identity adversary is trained to minimize  $L_{\text{identity}} = \text{cross\_entropy}(\text{identity\_label}, A(E(\text{raw\_window})))$ . The encoder is trained with a gradient-reversal or alternating-update objective:

$$L_{\text{encoder}} = L_{\text{task}}(T(E(\text{raw\_window})), \text{task\_label}) + \lambda_{\text{sparse}} * \text{sparsity\_penalty}(E(\text{raw\_window})) + \lambda_{\text{quant}} * \text{quantization\_penalty}(E(\text{raw\_window})) - \lambda_{\text{recon}} * L_{\text{decoder}} - \lambda_{\text{identity}} * L_{\text{identity}} + \lambda_{\text{metadata}} * \text{metadata\_leakage\_score}(E(\text{raw\_window}))$$

In another procedure, the encoder is accepted only if a held-out attack suite satisfies all configured acceptance criteria, such as normalized reconstruction error above a threshold, identity-classification accuracy no better than a permitted baseline margin, mutual-information estimate below a threshold, and task accuracy or task agreement above a target threshold. Example metrics may include normalized mean absolute reconstruction error, spectral reconstruction error, structural similarity for event rasters, identity AUC, membership-inference advantage, mutual-information estimate, active-position leakage score, task accuracy, and encrypted/plaintext score agreement.

For an event dataset such as N-MNIST or another rights-cleared event stream, a test protocol may compare dense raw windows, unsorted sparse events, spatial-sorted sparse events, padded sparse batches, and dense encrypted windows. The protocol may report task agreement, encrypted operation counts, ciphertext bytes, runtime, active-event density, reconstruction-attack success, and metadata leakage for each representation.

## F.9 Metadata Leakage Controls

Even when latent values are encrypted, metadata may reveal sensitive information. Metadata may include active channel positions, event timing, packet frequency, window density, session length, model selection, confidence scores, device identifiers, calibration state, and error flags. The system may reduce metadata leakage by padding sparse events, batching windows, coarsening timing, encrypting active indices, using private information retrieval, sending fixed-rate traffic, applying dummy traffic, or routing high-risk sessions to local-only inference.

## F.10 Split-Device and Trust-Zone Architecture

The system may be divided among:

- A sensor or headset that acquires raw telemetry.
- A phone, wearable hub, implant controller, laptop, or local server that performs preprocessing, encoding, compression, encryption, and policy checks.
- A cloud server, local enterprise server, hospital server, research server, or decentralized compute network that performs encrypted inference.
- A client application, clinician workstation, assistive control system, or user device that receives and decrypts authorized outputs.

The raw neural telemetry may remain in the acquisition device or edge hub. Secret keys may remain in a user-controlled secure element, mobile operating-system key store, hardware security module, local server, or threshold key arrangement. Remote compute may receive ciphertext, public model metadata, public parameters, and policy tokens, but not raw telemetry.

### **F.11 Adaptive Routing**

The system may decide whether inference is local, encrypted remote, enclave-protected remote, MPC-based, or deferred. For a safety-critical prosthetic control loop, a local model may run immediately while encrypted remote inference provides secondary verification. For a research analytics workflow, many encrypted embeddings may be batched for multi-user inference. For a low-bandwidth setting, a smaller embedding may be transmitted. For a high-risk cognitive-state task, metadata may be padded or inference may remain local.

### **F.12 Audit, Policy, and Consent Metadata**

The relay may generate an audit record identifying the sensor type, encoder version, compression policy, cryptographic scheme, privacy mode, inference target, authorization basis, retention policy, and result-handling rule. The audit record may avoid raw telemetry while preserving enough information to verify that the computation used an approved privacy boundary.

### **F.13 Hardware Co-Design**

The system may be implemented as a co-designed hardware and cryptographic pipeline. A neural acquisition device, headset controller, implant controller, neuromorphic front end, FPGA, ASIC, or digital signal processor may perform event detection, event binning, spatial sorting, thresholding, and active-position extraction. A local CPU, GPU, secure enclave, mobile processor, or edge server may perform policy selection, active-value encryption, key handling, audit-record generation, and optional local fallback inference. A remote CPU, GPU, accelerator, or server cluster may perform encrypted inference using OpenFHE, SEAL, TFHE-rs, Concrete, or another reviewed encrypted-computation library.

In one hardware-oriented embodiment, an FPGA or ASIC spike sorter emits a fixed schema of sorted sparse events. The local edge CPU encrypts the event values under a BFVrns parameter set and sends permitted active positions plus encrypted values to a remote scorer. The remote scorer applies public non-negative integer model weights and returns encrypted class scores. The local endpoint decrypts scores and applies safety, consent, and task-policy checks before displaying a result or issuing a control signal.

This hardware split improves the technological character of the system by making the privacy boundary a concrete signal-processing and cryptographic boundary: raw sample order, raw electrode identifiers, raw waveform traces, subject/session references, and active feature values remain local, while only policy-approved sparse descriptors and ciphertexts leave the device.

## **G. Example Embodiments**

### **G.1 EEG Attention Classifier**

An EEG headset samples multiple channels of scalp potentials. A mobile phone receives the stream, applies filtering and artifact rejection, and segments the stream into 250 ms overlapping windows. A temporal encoder produces a 64-dimensional low-bit latent embedding configured for BFV arithmetic. The phone encrypts the embedding and transmits it to a server. The server applies an encrypted linear attention classifier and returns encrypted class scores. The phone decrypts the scores and displays an authorized attention-state output.

### **G.2 Implanted Motor-Intent Relay**

An implanted array or implant controller records spike events and local field potential features. An edge hub bins spike counts into short windows and produces a sparse latent motor-intent embedding. The embedding values are encrypted, while active positions are padded to a fixed batch size. A remote inference service computes encrypted motor-intent scores. The local controller decrypts the result and applies local safety checks before issuing a prosthetic-control command.

### **G.3 fNIRS Workload Monitor**

A wearable fNIRS device measures hemodynamic features over longer windows. A local encoder

compresses oxygenated and deoxygenated hemoglobin patterns into a task-specific embedding. A compression controller increases window length and lowers latent precision when bandwidth is constrained. Encrypted inference classifies workload state without sending raw fNIRS traces to the remote service.

#### **G.4 Multi-User Research Analytics**

Multiple participants use local devices that generate encrypted neural embeddings under participant-specific or threshold-managed keys. A research server performs batch encrypted inference or aggregate statistics over embeddings. Differential privacy is applied to released aggregates. Raw participant telemetry remains local, and model updates are exchanged using secure aggregation.

#### **G.5 Clinical Edge Server**

A hospital workstation receives neural telemetry from a clinical neural device and performs local encoding. Encrypted embeddings are transmitted to a local hospital server or cloud enclave for specialized inference. The result is decrypted only by the clinical workstation or a threshold group authorized by hospital policy.

#### **G.6 Spatial-Sorted BFVrns Motor-Intent Scorer**

An implanted array, headset, or event-derived neural device emits sparse activity over short windows, such as 50 ms windows. A local spatial spike sorter converts raw event order and raw electrode identifiers into a canonical event grid. For an 8 by 8 feature grid, a representative event window may contain 64 possible positions and 18 active positions. The local edge device publishes or otherwise permits active neuron/time positions according to a privacy policy, encrypts only the 18 active feature values, and withholds raw neural samples, raw electrode identifiers, raw sample ordering, device identifiers, and subject/session references.

A remote encrypted inference engine uses a BFVrns integer context to compute class scores under the contract  $\text{scores} = W \times \text{active\_features} + \text{bias}$ . Public model weights are multiplied by encrypted active feature values and accumulated into encrypted class-score ciphertexts. The remote engine returns encrypted class scores to the local endpoint. The local endpoint decrypts the scores and applies authorized result handling, such as local safety checks for an assistive-control command.

In a prototype implementation, the BFVrns integration target uses a plaintext modulus of 65537, multiplicative depth 1, batch size 1, non-negative integer feature values, public non-negative integer model weights, public bias values, encrypted active feature values, public active-position descriptors, and client-side score decryption. These parameters are an implementation and benchmark target, not a production security claim.

### **H. Alternative Embodiments**

The latent encoder may be located in a headset, implant controller, mobile phone, smart watch, laptop, home server, hospital gateway, vehicle, or secure enclave. The encrypted inference engine may be located in a cloud server, local server, peer device, decentralized compute network, hardware accelerator, secure enclave, or multiparty protocol.

The system may use raw-to-latent autoencoders, manually engineered features, spike encoders, wavelet transforms, learned spectral features, event cameras combined with neural telemetry, multimodal biosignal encoders, or language-model-compatible tokenizers. The system may perform inference with linear models, shallow neural networks, SNN-inspired classifiers, polynomial neural networks, decision trees, encrypted lookup tables, or hybrid local and encrypted remote models.

The encryption may be applied to the entire embedding, selected dimensions, active values only, active indices and values, model weights, result scores, audit tokens, or combinations thereof. Some embodiments may combine FHE for arithmetic layers, TEE for model orchestration, MPC for key custody, and differential privacy for aggregate model updates.

The output may be a user-facing classification, a command, a confidence score, a clinical alert, a research label, an encrypted report, a proof of inference, a policy decision, or a local action. A result may remain encrypted until opened by an authorized party.

### **I. Use Cases**

Use cases include:

- Privacy-preserving EEG attention-state inference.
- Encrypted motor-intent classification for prosthetic or cursor control.
- Encrypted fatigue or workload detection for safety-critical environments.
- Assistive communication using local speech-intent embeddings and encrypted remote scoring.
- Neurofeedback where raw signals stay local and only encrypted features are processed externally.
- Privacy-preserving clinical triage or neurodiagnostic inference.
- Secure research collaboration across institutions.
- Multi-user encrypted batch inference for model validation.
- Federated learning over neural embeddings with secure aggregation.
- Consumer BCI applications in which application providers do not receive raw neural telemetry.
- Local-first gaming, productivity, and human-computer interaction using protected cognitive-state features.

## J. Experimental and Implementation Examples

### J.1 Reference Data Flow

An implementation may use the following data flow:

1. Acquire raw neural telemetry from one or more sensor channels.
2. Segment telemetry into windows using time, event, or task boundaries.
3. Apply preprocessing and signal-quality estimation.
4. Generate a compressed latent embedding locally.
5. Evaluate reconstruction-risk and task-utility metrics.
6. Select an adaptive compression and encryption policy.
7. Encrypt the latent embedding.
8. Transmit encrypted embedding and permitted metadata to an inference engine.
9. Perform encrypted inference.
10. Return encrypted scores or an authorized result.
11. Decrypt or consume the result only at an authorized endpoint.
12. Store an audit record that excludes raw neural telemetry.

### J.2 Example Pseudocode

```

window = segment(neural_stream, policy.window_ms, policy.overlap)
quality = estimate_signal_quality(window)
task = get_inference_target()

compression_policy = controller.select(
    signal_quality=quality,
    bandwidth=uplink_state,
    encryption_budget=he_context.noise_budget,
    entropy=estimate_entropy(window),
    task_type=task,
    privacy_budget=user_policy.privacy_budget
)

embedding = edge_encoder.encode(window, compression_policy)
embedding = privacy_filter.reduce_reconstruction_risk(embedding, user_policy)
ciphertext = crypto.encrypt_embedding(embedding, he_context.public_key)

encrypted_scores = relay.send_for_encrypted_inference(
    ciphertext,
    model_id=task.model_id,
    policy=compression_policy.public_descriptor
)

result = authorized_endpoint.decrypt_or_consume(encrypted_scores)

```

### J.3 Homomorphic Linear Classifier Example

In one implementation, a compressed embedding vector  $x$  is encrypted under a BFV or BGV scheme. A model matrix  $W$  and bias vector  $b$  define class scores according to  $\text{scores} = Wx + b$ . The server computes encrypted dot products using plaintext-ciphertext multiplication and encrypted addition.

The server returns encrypted scores to the client. The client decrypts the scores and selects a class locally.

#### J.4 CKKS Approximate Inference Example

In another implementation, a floating-point or fixed-point latent embedding is packed into CKKS ciphertext slots. The encrypted inference engine applies a low-depth polynomial network with approximate activations. Rescaling and rotations are selected based on a noise budget. The result is an encrypted score vector for decryption by an authorized endpoint.

#### J.5 TFHE Threshold Example

In another implementation, the embedding is binarized into thresholded neural event features. A TFHE-style engine evaluates Boolean logic, lookup tables, or threshold gates corresponding to a cognitive-state classifier. This implementation may be suitable for very low-bit event representations.

#### J.6 MPC Example

In another implementation, the local device secret-shares the latent embedding among multiple non-colluding servers. The servers jointly evaluate an inference function without any one server seeing the embedding. The result is reconstructed only for an authorized endpoint.

#### J.7 Quantitative Prototype Comparison

The following prototype data comes from the repository benchmark artifact for a synthetic 8 by 8 event window. It is included as support for the type of comparative evidence the system reports. It is not a clinical result and is not a production cryptographic security claim.

|  | Representation or mode                              | Feature slots encrypted | Active events | Encryptions | Scalar multiplies | Adds | Decryptions | Relative scalar multiplies |
|--|-----------------------------------------------------|-------------------------|---------------|-------------|-------------------|------|-------------|----------------------------|
|  | Public active positions                             | 18                      | 18            | 20          | 36                | 36   | 2           | 1.00x                      |
|  | Public active neuron positions + encrypted features | 18                      | 18            | 20          | 36                | 36   | 2           | 1.00x                      |
|  | Padded sparse batches                               | 32                      | 18            | 34          | 64                | 64   | 2           | 1.78x                      |
|  | Dense encrypted windows                             | 64                      | 18            | 66          | 128               | 128  | 2           | 3.56x                      |

For this representative window, the active-event density is 18/64, or 28.125 percent. The public-active-neuron-positions mode uses 20 encryptions and 36 scalar multiplies, while the dense encrypted window mode uses 66 encryptions and 128 scalar multiplies. On this window, the sparse public-position mode therefore reduces scalar multiplications and additions by 92 out of 128 operations, or about 71.875 percent, and reduces encryptions by 46 out of 66, or about 69.7 percent, compared with encrypting every dense feature slot. A toy additive benchmark artifact measured 1.897 ms and 200 ciphertext bytes for the demonstration path, while expressly marking the security parameters as research-only. A native OpenFHE BFVrns implementation target is provided for real-library benchmarking when OpenFHE is installed.

#### J.8 Adaptive Privacy-Mode Selector Pseudocode

```
inputs:
  active_count
  feature_count
  signal_quality
  bandwidth
  latency_target
  encryption_budget
  reconstruction_risk
  metadata_leakage
  task_sensitivity

density = active_count / feature_count

if reconstruction_risk > policy.max_reconstruction_risk:
  return LOCAL_ONLY

if metadata_leakage > policy.max_metadata_leakage:
  if encryption_budget.permits_dense_window:
    return DENSE_ENCRYPTED_WINDOW
  return PADDED_SPARSE_BATCH
```

```

if task_sensitivity == HIGH and policy.hide_exact_event_count:
    return PADDED_SPARSE_BATCH

if density <= policy.sparse_density_limit
    and bandwidth >= policy.min_sparse_bandwidth
    and latency_target.allows_remote:
    return PUBLIC_ACTIVE_POSITIONS_ENCRYPTED_FEATURES

if encryption_budget.permits_dense_window:
    return DENSE_ENCRYPTED_WINDOW

return LOCAL_ONLY_OR_DEFERRED

```

The selector may emit a policy record including selected representation, padding bucket, encryption scheme, parameter set identifier, metadata fields allowed outside the trusted boundary, result-handling rule, and reason codes. The policy record may be stored without raw neural telemetry.

## J.9 OpenFHE BFVRns Best-Mode Implementation Parameters

The current native integration target is `prototype/openfhe/openfhe_linear_demo.cpp` with `prototype/openfhe/CMakeLists.txt`. The implementation uses OpenFHE BFVRns exact integer arithmetic for a sparse sorted-event linear scorer.

```

| Parameter | Example value or rule |
|---|---|
| Event representation | Spatial-sorted sparse events |
| Feature shape | 8 by 8 feature grid |
| Feature count | 64 |
| Active event count | 18 in the representative synthetic window |
| Privacy mode | Public active neuron positions + encrypted feature values |
| Score contract | scores = W x + bias |
| Score domain | Non-negative integers |
| Classes | normal, anomaly in the prototype target |
| Scheme | OpenFHE BFVRns |
| Plaintext modulus | 65537 |
| Multiplicative depth | 1 |
| Batch size | 1 |
| Encrypted fields | Active feature values and class-score ciphertexts |
| Public fields in the fastest sparse mode | Active neuron positions, feature shape, public model weights, public bias |
| Withheld fields | Raw neural samples, raw electrode identifiers, raw sample timestamp order, device identifiers, local subject/session references |
| Expected prototype scores | normal: 9, anomaly: 51 |
| Expected prototype classification | anomaly |
| Native build commands | cmake -S prototype/openfhe -B build/openfhe; cmake --build build/openfhe; build/openfhe/openfhe_linear_demo |

```

The OpenFHE target should be treated as the best currently documented implementation mode for the narrow sorted-event BFVRns scorer. Production deployment would require reviewed security parameters, side-channel review, key-management design, real neural or event-dataset validation, and performance measurements on the target hardware.

## J.10 Privacy Metrics and Attack Test Protocol

A reconstruction-resistance test may run the following attacks against each candidate representation:

1. A raw-window decoder trained to reconstruct waveform or event-raster content from embeddings or sorted sparse events.
2. An identity classifier trained to infer user, session, device, or calibration identifiers.
3. A sensitive-attribute classifier trained to infer task-irrelevant cognitive, physiological, or demographic labels where such testing is lawful and ethically approved.
4. A metadata-only classifier trained on active-position descriptors, event counts, timing patterns, padding bucket sizes, and packet timing.
5. A membership or linkage attack that attempts to determine whether two encrypted-relay records came from the same user or session.

For each attack, the system may report task utility, reconstruction loss, attack accuracy, attack

AUC, mutual-information estimate, event-count leakage, position-pattern leakage, and whether the result passes a configured privacy policy. An encoder or privacy mode may be rejected if task utility is adequate but reconstruction or linkage risk exceeds the threshold.

## K. Security and Privacy Properties

The disclosed architecture may provide one or more of the following properties:

- Raw neural telemetry remains within a trusted local boundary by default.
- The remote processor receives encrypted latent embeddings rather than raw signals.
- Compression before encryption reduces bandwidth and encrypted operation count.
- Reconstruction-resistant encoding reduces raw-signal reconstruction risk.
- Identity-obfuscating encoding reduces user re-identification risk from embeddings.
- Adaptive compression balances accuracy, privacy, bandwidth, and encryption cost.
- Metadata leakage is measured and reduced through padding, batching, timing controls, or encrypted indices.
- Secret keys remain with an authorized client, local device, secure hardware, or threshold key group.
- Audit records describe privacy boundaries and cryptographic choices without storing raw telemetry.
- Hybrid cryptographic modes allow deployment across FHE, SHE, MPC, TEE, differential privacy, secure enclaves, and conventional encrypted transport.

No particular cryptographic primitive is required in all embodiments. The system may be cryptographically agile so that encryption schemes, parameter sets, and execution modes can be replaced as performance, security standards, and deployment constraints evolve.

## L. Examiner-Facing Prior-Art Distinctions

This section identifies candidate prior-art distinctions to preserve during later nonprovisional drafting. It is not an information disclosure statement and should be verified by counsel before filing any nonprovisional claims.

Candidate HE-SNN references show that homomorphic encryption has already been applied to spiking neural networks and sparse neural-network inference. For example, PrivSpike uses CKKS for private inference of deep SNNs and reports encrypted SNN inference on MNIST, CIFAR-10, Neuromorphic MNIST, and CIFAR-10 DVS. Nikfam et al. describe BFV-based privacy-preserving SNN inference using Pyfhel and FashionMNIST-style models. FHE-DiSNN describes homomorphic evaluation of discretized SNN operations, including protected firing and reset behavior. SpENCNN describes HE-aware sparsity and packing for faster encrypted neural-network inference.

The narrow embodiment disclosed here should therefore not be framed merely as "using HE on an SNN" or "using sparsity to speed encrypted neural-network inference." The stronger distinction is a local trusted-boundary relay for neural telemetry in which raw event order, raw electrode identifiers, raw waveform material, subject/session references, and active feature values remain local; a spatial spike sorter emits a policy-governed sparse event representation; active positions may be public, padded, coarsened, randomized, or encrypted according to metadata-leakage policy; active feature values are encrypted before external transmission; and the remote scorer evaluates a bounded-depth encrypted linear contract over that representation.

In relation to HE-SNN systems, this disclosure emphasizes:

- local neural telemetry boundary before remote encrypted compute;
- sorted sparse event readiness as a relay representation rather than a full encrypted SNN model as the inventive center;
- public-active-position plus encrypted-active-value mode as one explicit privacy/performance point in a selectable family;
- metadata-leakage controls that can move between public active positions, padded sparse batches, and dense encrypted windows;
- reconstruction-resistance testing and identity/linkage attack testing for neural embeddings or sorted event representations; and
- audit records that document encoder version, sorter version, privacy mode, cryptographic scheme, and result handling without retaining raw telemetry.

In relation to generic HE neural-network acceleration, this disclosure ties the compression and sparse-event choices to neural telemetry, BCI privacy, local raw-signal retention, and task-specific cognitive-state inference rather than generic image or cloud ML inference alone.

In relation to blockchain storage and access-control references, this disclosure is not centered on storing encrypted neural records, smart-contract permissioning, off-chain address retrieval, or decentralized key-release workflows. Those functions may be optional infrastructure. The inventive center is local transformation and privacy-mode selection before encrypted neuroinference.

## M. Claim-Like Embodiments

The following embodiments are written in claim-like form for later conversion into formal claims:

1. A method comprising receiving spike-event or event-derived neural telemetry at a local device, segmenting the telemetry into a time window, sorting the time window into a sparse event representation using a local spatial spike sorter, encrypting active feature values from the sparse event representation, transmitting encrypted active feature values and a permitted position descriptor to a remote inference engine, performing bounded-depth encrypted inference on the encrypted active feature values, and outputting an encrypted or authorized cognitive-state classification without exposing raw neural telemetry or plaintext active feature values to the remote inference engine.
2. The method of embodiment 1, wherein the neural telemetry comprises EEG, ECoG, fNIRS, MEG, implanted-array telemetry, wearable BCI telemetry, local field potentials, spike events, or combinations thereof.
3. The method of embodiment 1, wherein the permitted position descriptor includes active neuron positions, channel positions, time bins, spatial bins, or flattened feature indices and excludes the active feature values.
4. The method of embodiment 1, wherein the encrypted inference comprises a BFVrns depth-1 linear score contract.
5. The method of embodiment 4, wherein the score contract is  $\text{scores} = W \times \text{bias}$ .
6. The method of embodiment 1, wherein the permitted position descriptor is selected between public active positions, padded sparse batches, and dense encrypted windows based on a privacy policy.
7. The method of embodiment 1, wherein the sparse event representation, compressed latent embedding, or active feature vector is reconstruction-resistant.
8. The method of embodiment 1, wherein the local encoder is trained using an adversarial decoder that attempts to reconstruct raw neural telemetry from the sparse event representation or a compressed latent embedding.
9. The method of embodiment 1, wherein encrypting comprises applying fully homomorphic encryption, somewhat homomorphic encryption, leveled homomorphic encryption, secure multiparty computation, trusted execution environment protection, differential privacy, secure enclave processing, hybrid encryption, or combinations thereof.
10. The method of embodiment 1, further comprising adapting sparse-event padding, latent dimensionality, quantization precision, sparsity level, packing layout, local-vs-remote routing, or encryption scheme based on encryption cost, bandwidth, entropy, task type, signal quality, reconstruction risk, or metadata leakage.
11. A system comprising a neural acquisition device, a local edge device comprising a spatial spike sorter configured to generate sorted sparse events from neural telemetry, an encryption module configured to encrypt active feature values, a relay interface configured to transmit encrypted active feature values and permitted position descriptors, and an encrypted inference engine configured to perform inference on the encrypted active feature values.
12. The system of embodiment 11, wherein the spatial spike sorter is implemented in a wearable headset, implant controller, FPGA, ASIC, neuromorphic front end, digital signal processor, mobile phone, local server, secure enclave, or edge hub.
13. The system of embodiment 11, wherein the encrypted inference engine is located in a cloud server, local server, hospital server, decentralized compute network, secure enclave, or



multiparty computation environment.

14. A non-transitory computer-readable medium storing instructions that, when executed by one or more processors, cause the processors to receive neural telemetry, segment the neural telemetry into windows, generate sorted sparse events or compressed latent embeddings, select an adaptive compression policy, encrypt active feature values or latent values, transmit encrypted values to an inference engine, and receive encrypted or authorized inference results.

## **N. Abstract**

An encrypted neural embedding relay receives neural telemetry and processes the telemetry locally to generate compressed latent embeddings or sorted sparse event representations. In a preferred embodiment, a spatial spike sorter located within a trusted local boundary converts spike-event or event-derived neural telemetry into active position descriptors and active feature values. Active feature values are encrypted before external transmission, while active position descriptors are public, padded, coarsened, encrypted, or withheld according to a privacy policy. A remote encrypted inference engine may evaluate a depth-1 linear score contract over the encrypted active feature values using BFV, BFV<sub>rns</sub>, BGV, TFHE-style integer operations, or related privacy-preserving computation. An adaptive compression controller may select latent dimensionality, quantization, sparsity, padding, packing, local-vs-remote routing, and encryption parameters based on signal quality, bandwidth, encryption budget, entropy, task type, metadata leakage, reconstruction risk, and privacy budget. The architecture prevents exposure of raw neural telemetry and plaintext active feature values to remote processors while enabling cognitive-state, motor-intent, attention-state, medical, or neurodiagnostic inference from protected representations.

# ENER Drawings and Figure Descriptions

Title: Encrypted Neural Embedding Relay for Privacy-Preserving Neuroinference

This document provides simple architecture diagrams and patent-style descriptions for the provisional application. The Mermaid source files are stored in patent/figures/ and mirrored in patent/ENER\_figures/.

## Figure 1 - Overall ENER Pipeline

```
flowchart LR
    A[Neural sensor] --> B[Edge encoder]
    B --> C[Compressed latent embedding]
    C --> D[Encryption module]
    D --> E[Relay interface]
    E --> F[Encrypted inference engine]
    F --> G[Encrypted or authorized output]
```

Figure 1 illustrates an end-to-end encrypted neural embedding relay pipeline. Neural telemetry is acquired by a neural sensor and provided to an edge encoder located within a trusted local boundary. The edge encoder converts raw or preprocessed telemetry into a compressed latent embedding. The compressed latent embedding is encrypted before transmission through a relay interface. An encrypted inference engine performs selected inference operations over the encrypted latent representation and returns an encrypted result or an authorized output. Raw neural telemetry is not exposed to the remote inference engine.

## Figure 2 - Split-Device Architecture

```
flowchart LR
    H[BCI headset or neural acquisition device] --> P[Phone or edge hub]
    P -->|local preprocessing and latent encoding| E[Encryption module]
    E -->|encrypted embedding| S[Encrypted compute server]
    S -->|encrypted result| C[Client application]
    C -->|authorized decryption or display| U[User or authorized recipient]
```

Figure 2 illustrates a split-device architecture. A BCI headset, implanted-device controller, wearable sensor, or other neural acquisition device provides telemetry to a phone, local hub, or edge processor. The phone or edge hub performs preprocessing, segmentation, latent encoding, adaptive compression, encryption, and policy checks. An encrypted compute server receives encrypted embeddings and computes inference results without receiving raw neural telemetry. A client application receives the encrypted result and decrypts or displays the result only when authorized.

## Figure 3 - Adaptive Latent Compression Controller

```
flowchart LR
    SQ[Signal quality] --> C[Adaptive compression controller]
    BW[Bandwidth state] --> C
    EB[Encryption budget] --> C
    TT[Inference target] --> C
    EN[Entropy estimate] --> C
    PB[Privacy budget] --> C
    C --> P[Compression policy]
    P --> LD[Latent dimensionality]
    P --> Q[Quantization]
    P --> SP[Sparsity and padding]
    P --> PK[Ciphertext packing]
    P --> R[Local or remote routing]
```

Figure 3 illustrates an adaptive latent compression controller. The controller receives signal-quality information, bandwidth state, encryption-budget information, inference target, entropy estimate, and privacy-budget constraints. Based on these inputs, the controller outputs a compression policy that may control latent dimensionality, quantization precision, sparsity, padding, ciphertext packing, and local or remote routing. The controller allows the system to

balance task accuracy, reconstruction resistance, bandwidth use, and encrypted-compute cost.

## Figure 4 - Reconstruction-Resistant Encoder Training

```
flowchart LR
    R[Raw neural signal] --> E[Encoder]
    E --> Z[Latent embedding]
    Z --> T[Task classifier]
    T --> O[Task output]
    Z --> A[Adversarial decoder]
    A -. blocked or penalized .-> X[Attempted raw-signal reconstruction]
```

Figure 4 illustrates reconstruction-resistant encoder training. A raw neural signal is provided to an encoder that generates a latent embedding. A task classifier is trained to predict a target output from the latent embedding. An adversarial decoder attempts to reconstruct the raw neural signal or sensitive attributes from the embedding. Training penalizes successful reconstruction while preserving task utility, thereby producing an embedding that is more resistant to raw-signal reconstruction or identity leakage.

## Figure 5 - Encrypted Cognitive-State Classifier

```
flowchart LR
    Z[Encrypted latent vector] --> L1[Encrypted model layer]
    L1 --> L2[Encrypted accumulation or activation]
    L2 --> L3[Encrypted score layer]
    L3 --> R[Encrypted classification result]
    R --> D[Authorized decryption endpoint]
    D --> O[Cognitive-state output]
```

Figure 5 illustrates an encrypted cognitive-state classifier. An encrypted latent vector is provided to encrypted model layers that may include linear projections, polynomial activations, threshold logic, encrypted accumulation, encrypted lookup tables, or other privacy-preserving operations. The classifier produces an encrypted classification result, encrypted score vector, or encrypted confidence value. An authorized decryption endpoint decrypts or consumes the result and outputs a cognitive-state classification, motor-intent command, attention-state estimate, medical or neurodiagnostic label, or other authorized result.

## Figure 6 - Spatial Sparse-Event BFVrns Scorer

```
flowchart LR
    A[Spike-event neural telemetry] --> B[Local spatial spike sorter]
    B --> C[Active position descriptor]
    B --> D[Active feature values]
    C --> E[Relay policy gate]
    D --> F[Encrypt active feature values]
    E --> G[Permitted positions]
    F --> H[Encrypted active values]
    G --> I[BFVrns depth-1 linear scorer]
    H --> I
    I --> J[Encrypted class scores]
    J --> K[Authorized local decryptor]
```

Figure 6 illustrates a preferred sorted sparse-event embodiment. Spike-event or event-derived neural telemetry is processed by a local spatial spike sorter within a trusted boundary. The sorter separates active position descriptors from active feature values. A relay policy gate determines whether active positions are public, padded, coarsened, encrypted, or withheld. Active feature values are encrypted before external transmission. A BFVrns depth-1 linear scorer computes encrypted class scores using the permitted positions and encrypted active values, and an authorized local decryptor decrypts or consumes the result.